Atividade A1 – Universidade São Judas Tadeu
Exploração Digital

Nome: Raphael Souza Silva

Q1. Analógico vs. Digital

Um sinal analógico é contínuo e pode assumir infinitos valores dentro de um intervalo. Um exemplo é o som captado por um microfone analógico ou a temperatura medida por um termômetro de mercúrio.

Um sinal digital utiliza valores discretos, geralmente representados por 0 e 1, que correspondem aos bits utilizados pelos computadores.

Exemplos de aplicações:

- Analógico: rádio AM/FM tradicional, toca-discos e termômetro de mercúrio.
- Digital: computadores, celulares, câmeras digitais e relógios digitais.

Os computadores utilizam sinais digitais porque seus circuitos eletrônicos conseguem representar e processar os estados binários de maneira confiável. Isso facilita o armazenamento, o processamento e a transmissão de informações.

Q2. Organização dos computadores – Modelo de Von Neumann

O modelo de Von Neumann é uma arquitetura de computadores em que dados e instruções ficam armazenados na mesma memória.

Seus principais componentes são:

- Unidade de Controle (UC): coordena a execução das instruções e controla o funcionamento dos componentes.
- Unidade Lógica e Aritmética (ULA): realiza operações matemáticas e lógicas.
- Memória: armazena dados e instruções dos programas.
- Entrada: permite inserir informações, como por meio do teclado e do mouse.
- Saída: apresenta os resultados, como na tela ou na impressora.
- Registradores: armazenam temporariamente dados, endereços e instruções durante o processamento.

Esses componentes interagem por meio de barramentos que transportam dados, endereços e sinais de controle. A CPU busca uma instrução na memória, interpreta seu significado e a executa, repetindo esse processo continuamente.

Q3. Ciclo de instrução

A instrução `ADD R1, A, B` representa, nesta atividade, a soma dos valores de `A` e `B`, com o resultado armazenado no registrador `R1`.

A execução ocorre nas seguintes etapas:

1. Busca (Fetch): a unidade de controle busca a instrução na memória utilizando o contador de programa (PC).
2. Decodificação (Decode): a unidade de controle interpreta a instrução e identifica os operandos `A` e `B` e o destino `R1`.
3. Busca dos operandos: os valores de `A` e `B` são obtidos dos locais em que estão armazenados.
4. Execução (Execute): a ULA realiza a soma dos dois valores.
5. Escrita do resultado (Write-back): o resultado é armazenado no registrador `R1`.
6. Atualização do PC: o contador de programa é atualizado para indicar a próxima instrução, salvo quando ocorre um desvio.

Para a instrução `MOV R1, A`, o processo é semelhante, mas não há soma: o valor de `A` é copiado para `R1`.

$$R1 \leftarrow A$$

No caso da instrução ADD:

$$R1 \leftarrow A + B$$

A sintaxe exata pode variar conforme a arquitetura do processador, mas essas expressões representam o funcionamento solicitado.

Q4. Pipeline

O pipeline é uma técnica de processamento que divide a execução das instruções em etapas. Dessa forma, várias instruções podem estar sendo processadas ao mesmo tempo, cada uma em uma etapa diferente.

As etapas mais comuns são:

1. Busca: procura a instrução na memória.
2. Decodificação: interpreta a instrução.
3. Execução: realiza a operação solicitada.
4. Acesso à memória: lê ou grava dados, quando necessário.
5. Escrita: registra o resultado.

Vantagens:

- Aumenta a quantidade de instruções processadas por unidade de tempo.
- Melhora o aproveitamento dos componentes do processador.
- Pode aumentar o desempenho geral do computador.

Problemas potenciais:

- Dependência de dados: uma instrução precisa do resultado de outra.
- Conflitos estruturais: duas instruções precisam do mesmo recurso ao mesmo tempo.
- Desvios de execução: saltos condicionais podem exigir o descarte de instruções já iniciadas.

Q5. Conversão de bases numéricas

Tabela preenchida com as conversões correspondentes:

Decimal	Binário	Octal	Hexadecimal
4034	**111111000010**	**7702**	**FC2**
1978	**11110111010**	**3672**	**7BA**
484	**111100100**	**744**	**1E4**
2879	**101100111111**	**5477**	**B3F**

Exemplos dos cálculos:

- $FC2_{16} = 15 \times 256 + 12 \times 16 + 2 = 4034_{10}$
- $754_{8} = 7 \times 64 + 5 \times 8 + 4 = 484_{10}$
- $101100111111_{2} = 2879_{10}$

Para converter de binário para octal, agrupamos os bits de três em três. Para hexadecimal, agrupamos de quatro em quatro, completando com zeros à esquerda quando necessário.

Q6. Tabela verdade e circuito lógico

Uma tabela verdade apresenta todas as combinações possíveis das entradas e o resultado de uma expressão lógica. Para três variáveis (A, B e C), existem $2^3 = 8$ combinações.

a) Expressão S_1

Considerando a expressão da imagem:

$$S_1 = \bar{A}B\bar{C} + \overline{(\bar{B}C \oplus A) \cdot (AC + B)}$$

TABELA VERDADE

A	B	C	P1	P2	P3	P4	S1
0	0	0	0	0	0	1	1
0	0	1	0	1	0	1	1
0	1	0	1	0	1	1	1
0	1	1	0	0	1	1	1
1	0	0	0	1	0	1	1
1	0	1	0	0	1	1	1
1	1	0	0	1	1	0	0
1	1	1	0	1	1	0	0

b) Expressão S_2

Considerando a expressão da imagem:

$$S_2 = \overline{A + \bar{B} + \bar{C}} + ABC + (A \oplus C)$$

TABELA VERDADE

A	B	C	P1	P2	P3	S2
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	1	0	0	0	0	0
0	1	1	0	0	1	1
1	0	0	0	0	1	1
1	0	1	0	0	0	0
1	1	0	0	0	1	1

Q7. Para determinar a expressão lógica, analisamos cada porta do circuito, respeitando a ordem das operações. A bolinha na saída de uma porta indica negação (NOT).

a) Primeiro circuito

A expressão lógica correspondente é:

$$X = \overline{\overline{(A \cdot \bar{B})} \cdot \overline{(\bar{B} + C)} + (\bar{B} + C)}$$

Uma forma mais organizada de representar o circuito é definir os sinais intermediários:

$$Y_1 = \overline{A \cdot \bar{B}}$$

$$Y_2 = \overline{\bar{B} + C}$$

$$Y_3 = \overline{Y_1 \cdot Y_2}$$

$$X = \overline{Y_3 + \overline{Y_2}}$$

tabela verdade:

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

b) Segundo circuito

O circuito apresenta uma porta NOR na parte superior, um inversor na entrada C, uma porta AND no meio, uma porta NAND na parte inferior e uma porta OR na saída.

A expressão lógica é:

$$X = \overline{(A + \bar{B})} \cdot \bar{C} + \overline{A \cdot C}$$

Tabela verdade:

A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Conclusão: a análise dos circuitos lógicos permite transformar as portas em expressões booleanas e construir tabelas verdade para verificar o resultado de cada combinação de entradas.